

AI Runtime Platform Design

v1.0

Status: Design baseline

Role: Permanent reusable CV Platform subsystem

Scope: AI inference execution and runtime orchestration

> This document defines the reusable AI Runtime Platform. It is an internal platform design document, not a project-specific implementation guide.

Project-specific AI applications, models, rules, and camera assignments are configuration and composition concerns.

1. Purpose

The AI Runtime Platform is the reusable execution layer for computer-vision inference across all future CV projects.

Its purpose is to make the following capabilities build-once and reuse-many:

- DeepStream-based video inference
- TensorRT model execution
- CUDA/GPU execution
- DLA allocation where supported
- model serving and lifecycle
- pipeline construction and lifecycle
- batching
- serial, parallel, and cascaded model execution
- runtime scheduling
- GPU/resource allocation
- application-to-camera assignment
- inference metadata generation
- runtime health and recovery

The platform must allow a new project to select and configure existing AI applications and models without rebuilding the inference infrastructure.

The platform must not contain customer business rules, ROI/scene reasoning, evidence fusion, alerting, or external actions.

2. Position in the Permanent CV Platform

The canonical processing path is:

```text

Camera Platform

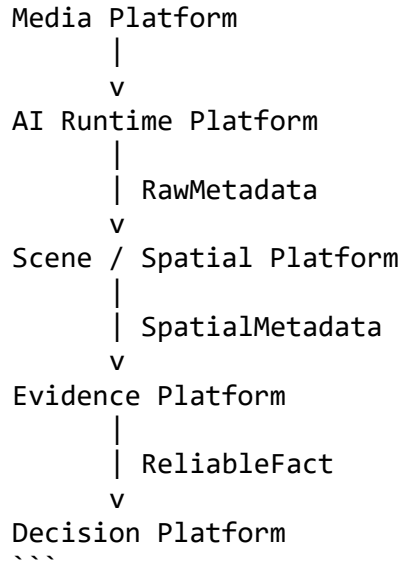
|

v

Camera Session Manager

|

v



The AI Runtime Platform is therefore responsible for transforming an available encoded/decoded video stream into **RawMetadata**. It does not decide whether a detection is a reliable fact and does not decide what business action should occur.

---

## 3. Architectural Boundary

### 3.1 Owns

The AI Runtime Platform owns:

- inference execution
- model runtime integration
- DeepStream integration
- TensorRT integration
- CUDA execution where required
- pipeline topology
- pipeline lifecycle
- inference batching
- runtime scheduling
- GPU/DLA resource allocation
- model loading and runtime lifecycle
- runtime model compatibility checks
- frame sampling/inference intervals
- AI application execution
- RawMetadata production
- runtime health and recovery for AI execution

### 3.2 Does not own

The AI Runtime Platform does **not** own:

- camera discovery
- ONVIF

- camera credentials management
- camera health as a device-management concern
- RTSP session ownership
- live-stream delivery to users
- recording policy
- ROI interpretation
- zones and line-crossing semantics
- camera calibration
- business rules
- evidence fusion
- identity/business authorization
- incidents
- notifications
- GPIO/relay actions

Hard rule:

```
> **AI Runtime knows inference. It does not know business meaning.**

```

## 4. Core Design Principle: AI Application, Not DeepStream Pipeline

The public platform abstraction is an **AI Application**.

DeepStream PGIE, SGIE, plugins, pipelines, and GStreamer elements are implementation details behind that abstraction.

Example AI Applications:

- Person Detection
- Face Detection
- Face Recognition
- Fire Detection
- Smoke Detection
- PPE Detection
- Vehicle Detection
- License Plate Recognition
- Pose Estimation
- Crowd Detection

A project should configure:

```
```text
```

```
Camera -> AI Application assignment
```

```
```
```

rather than manually constructing DeepStream graphs.

The runtime translates the application configuration into an executable runtime topology.

```

```

## 5. AI Application Model

An AI Application is a reusable definition of an inference capability.  
Conceptually:

```
```text
```

AI Application

```
|— application\_id
|— version
|— required\_models
|— execution topology
|— input requirements
|— output schema
|— inference interval / FPS policy
|— batching policy
|— resource requirements
|— scheduling policy
|— runtime capabilities
```

```
```
```

A project may assign the same AI Application to many cameras.

Example:

```
```text
```

Fire Detection

Camera 001

Camera 002

...

Camera 100

Person Detection

Camera 021

Camera 033

Camera 045

Camera 089

Face Recognition

Camera 021

Camera 038

Camera 045

Camera 055

Camera 078

```
```
```

The application definition is reusable; camera assignment is project configuration.

```

```

## 6. Runtime Topology

A single camera may run multiple independent AI applications simultaneously.  
Therefore the runtime must support:

```
```text
```

Camera

```
|
+--> Person Detection
|
```

```

+--> Face Recognition
|
+--> Fire Detection
|
+--> PPE Detection
|
+--> LPR
|
+--> Custom Application
...

```

The runtime must not assume one camera equals one model or one pipeline. It must be capable of choosing the most efficient execution topology for the configured applications.

7. Execution Strategies

The runtime must support at least these execution patterns.

7.1 Single-model execution

```

```text
Frame -> Model A -> RawMetadata
...

```

### 7.2 Serial execution

```

```text
Frame
|
v
Model A
|
v
Model B
|
v
Model C
...

```

Used when the output of one model is required by the next.

7.3 Parallel execution

```

```text
 +--> Model A --+
Frame -----+--> Model B ----+--> RawMetadata
 +--> Model C --+
...

```

Used for independent models.

### 7.4 Cascaded execution

```

```text
Frame
|

```

```

Detector
|
+---- person crops ----> Person Attribute Model
|
+---- face crops -----> Face Model
|
+---- vehicle crops ---> LPR
...

```

This corresponds naturally to detector/secondary-inference patterns such as PGIE/SGIE, but the platform abstraction must remain AI Application rather than PGIE/SGIE.

7.5 Shared inference

If multiple applications need the same expensive base inference, the runtime should be able to execute that inference once and share its output where contracts permit.

Example:

```

```text
Camera
|
Person Detector
|
+--> PPE
+--> Face
+--> ReID
+--> Person Attributes
...

```

This is a major optimization target for 100+ camera deployments.

---

## 8. PGIE / SGIE Policy

PGIE and SGIE are DeepStream implementation mechanisms, not permanent platform concepts.

The runtime may use:

- one PGIE
- multiple PGIEs
- one or more SGIEs
- multiple DeepStream pipelines
- shared inference branches
- independent pipelines

depending on the application graph and resource constraints.

The runtime must choose topology based on application requirements and resource efficiency rather than forcing every project into a single PGIE/SGIE structure.

---

## 9. Pipeline Orchestrator

The Pipeline Orchestrator converts configured AI Applications into executable runtime pipelines.

Responsibilities:

- resolve camera/application assignments
- resolve model dependencies
- construct pipeline topology
- determine execution strategy
- allocate runtime resources
- start pipelines
- stop pipelines
- restart failed pipelines
- apply configuration changes
- report runtime state

Conceptually:

```
```text
```

Configuration

|

v

AI Application Resolver

|

v

Topology Planner

|

v

Resource Planner

|

v

Pipeline Builder

|

v

DeepStream Runtime

```
```
```

---

## 10. Model Serving

Model serving is a first-class responsibility inside AI Runtime.

It must provide reusable infrastructure for:

- model loading
- model initialization
- model warm-up
- runtime compatibility checks
- TensorRT engine loading
- engine caching
- model lifecycle
- model unload
- model version selection
- failure handling
- resource accounting

Model semantics/configuration originate from the Configuration Platform; model artifacts are stored through the Storage Platform. AI Runtime owns their **execution lifecycle**.

The runtime must not assume that every model is identical. Different applications may require different model runtimes and execution characteristics.

---

## 11. Model Versioning

A model reference must be versioned.

Conceptually:

```
```text
```

```
model: person-detector
```

```
version: 4
```

```
artifact: ...
```

```
engine: ...
```

```
```
```

A deployment must be able to pin a model version.

Runtime upgrades must not silently replace an active model version.

Model replacement should support:

```
```text
```

```
old model -> validate new model -> warm new model -> switch -> retire old model
```

```
```
```

Hot replacement is a runtime capability, but exact implementation may depend on DeepStream/GStreamer/TensorRT constraints.

---

## 12. Runtime Scheduler

Scheduling is part of AI Runtime in v1.0.

The scheduler determines when configured AI applications execute.

It must support concepts such as:

- inference interval
- target FPS
- frame skipping
- priority
- camera/application scheduling
- resource-aware execution
- application enable/disable
- time-window scheduling when supplied as runtime configuration

Examples:

```
```text
```

```
Fire Detection      -> continuous/high priority
```

```
Person Detection    -> 15 FPS
```

```
Face Recognition    -> every 1 second
```

```
OCR                 -> every 5 frames
```

```
PPE                 -> only when person exists
```

```
```
```

The scheduler must distinguish **execution scheduling** from **business rules**.



Example:

```
```text
```

```
Run face recognition every second
    = runtime scheduling
```

```
Alert if unknown person is present after 22:00
    = business decision
```

```
```
```

```

```

## 13. GPU and Accelerator Resource Manager

The Resource Manager abstracts hardware allocation from individual applications.

It may manage:

- GPU devices
- GPU memory budgets
- CUDA execution resources
- DLA cores where supported
- decode-related constraints exposed to runtime
- accelerator-specific capabilities
- application resource requirements

Example intent:

```
```text
```

```
Fire detector -> DLA if supported
```

```
Person detector -> GPU
```

```
Face recognition -> GPU
```

```
```
```

The exact hardware mapping remains deployment-specific.

The platform must not hard-code one GPU topology for all deployments because the same reusable platform must run on Jetson, RTX PCs, and GPU servers.

```

```

## 14. Resource Allocation Principles

1. Do not allocate a resource merely because it exists.
2. Prefer sharing expensive inference when outputs are compatible.
3. Avoid unnecessary model duplication in GPU memory.
4. Avoid unnecessary decode duplication.
5. Prefer zero-copy paths where the selected runtime supports them.
6. Respect model memory requirements.
7. Preserve high-priority applications under contention.
8. Fail predictably when resources cannot satisfy the requested deployment.

```

```

## 15. Batching

The runtime should support batching where beneficial.

Batching may occur across:

- frames
- cameras
- inference requests

Batching policy must consider:

- latency
- throughput
- model requirements
- camera FPS
- number of cameras
- GPU utilization
- application priority

The runtime must not maximize batch size blindly. For surveillance, latency and predictable behavior can be more important than maximum throughput.

---

## 16. 100+ Camera Architecture

The runtime must be designed for many cameras and many-to-many application assignments.

Example:

```
```text
```

100 Cameras

```
|
```

```
+--> Fire Detection: 100
```

```
+--> Person Detection: 4
```

```
+--> Face Recognition: 5
```

```
+--> LPR: 12
```

```
+--> PPE: 23
```

```
+--> Other applications: N
```

```
```
```

The runtime must avoid creating unnecessary duplicate infrastructure.

It should conceptually maintain:

```
```text
```

Camera Registry

```
|
```

Application Assignments

```
|
```

Topology Planner

```
|
```

Model Instances / Shared Model Services

```
|
```

Runtime Pipelines

```
```
```

A model may be shared when compatible. A model must be instantiated separately when isolation, state, preprocessing, batching, or runtime constraints require it.

---

## 17. Application-to-Camera Assignment

Assignments are configuration, not source code.

Bad:

```
```python
if camera\_id == "cam33":
    run\_person()
```
```

Good:

```
```text
Configuration:
Camera 33 -> Person Detection
Camera 45 -> Person Detection + Face Recognition
```
```

The runtime consumes the assignment and builds the execution plan.

---

## 18. DeepStream Adapter

DeepStream is the primary inference/video-runtime implementation for v1.0. The DeepStream Adapter hides DeepStream-specific implementation details from the rest of the platform.

Responsibilities include:

- GStreamer/DeepStream pipeline construction
- nvinfer/nvinferserver/nvdspreprocess integration where applicable
- DeepStream metadata extraction
- TensorRT integration
- plugin configuration
- pipeline lifecycle operations
- DeepStream error translation

The rest of CV Platform should consume platform-level abstractions and contracts rather than DeepStream objects.

This follows Platform Contracts v1.0: platforms expose data, not implementation. The required external output is RawMetadata, not a DeepStream object.

---

## 19. NVIDIA DeepStream Coding Agent

NVIDIA's DeepStream Coding Agent/skill is an implementation aid for DeepStream-specific development.

It belongs in the CV Platform DeepStream knowledge/skill layer and may be invoked by AIOS for DeepStream tasks.

It is **not** part of the AI Runtime runtime architecture and is **not** a replacement for this platform's abstractions.

The split is:

```
```text
```

AIOS

- > orchestrates development
- > invokes DeepStream skill when needed

CV Platform

- > owns reusable DeepStream engineering knowledge
- > owns DeepStream Adapter/runtime code

AI Runtime

- > owns execution architecture
- ...
-

20. RawMetadata

The primary output of AI Runtime is **RawMetadata**.

It must follow Platform Contracts v1.0.

Conceptually:

```
```text
RawMetadata
├── camera_id
├── timestamp
├── frame_number
├── application_id
├── model_id
├── track_id
├── bounding_box
├── class
├── confidence
├── segmentation
├── keypoints
├── embedding
├── OCR
└── attributes
```
```

RawMetadata must not contain business decisions.

Examples of invalid AI Runtime output:

```
```text
person_is_intruder = true
fire_alarm = true
send_alert = true
```
```

Those meanings belong downstream.

21. Metadata Publication

AI Runtime publishes RawMetadata through the platform's defined contract/event mechanisms.

The runtime should support efficient metadata transport without coupling consumers to DeepStream internal metadata structures.

The exact transport mechanism must respect the Event Platform and Platform Contracts rather than creating a private application-specific messaging system.

22. Runtime Lifecycle

Every AI application execution should have a lifecycle similar to:

```text

UNRESOLVED

|

v

RESOLVING

|

v

PLANNED

|

v

STARTING

|

v

WARMING

|

v

RUNNING

|

+--> DEGRADED

|

+--> STOPPING

|

v

STOPPED

```

Failure should be explicit:

```text

RUNNING -> FAILED -> RECOVERING -> STARTING -> RUNNING

```

Exact state names may be refined during implementation, but lifecycle state must be observable and deterministic.

23. Failure Recovery

The runtime must handle at minimum:

- model load failure
- incompatible model/runtime
- pipeline construction failure
- GPU resource exhaustion
- DLA unavailable/incompatible
- DeepStream element failure
- pipeline crash

- repeated pipeline failure
- metadata publication failure
- configuration change failure

Recovery policy should distinguish:

```text

Transient failure

-> retry/restart

Persistent configuration failure

-> stop + report

Resource exhaustion

... -> degrade/replan where policy permits

```

The runtime must never silently claim that an application is running when its inference path is not operational.

24. Dynamic Configuration Changes

A project should not need to rebuild the application to change camera assignments or runtime parameters.

Examples:

```text

Camera 33:

Person Detection -> enabled

Camera 45:

Face Recognition -> disabled

Person FPS:

15 -> 10

```

The runtime should reconcile configuration changes into running pipelines where safely supported.

Possible strategy:

```text

Configuration change

|

v

Validate

|

v

Calculate runtime diff

|

v

Apply safely

|

v

Verify

|  
v  
Publish health/state  
...

Where a live change cannot be applied safely, the runtime should perform a controlled restart rather than corrupting pipeline state.

---

## 25. Health and Observability

AI Runtime must produce HealthStatus according to Platform Contracts v1.0. At minimum, health should expose:

- component state
- last successful inference
- model state
- pipeline state
- GPU utilization where available
- GPU memory usage where available
- inference latency
- effective FPS
- dropped/skipped frames
- queue/backpressure state
- metadata publication state
- restart count
- error information

Health is observability, not business decision-making.

---

## 26. Performance Metrics

The reusable runtime should make it possible to measure:

### Per camera

- input FPS
- processed FPS
- dropped FPS
- latency

### Per application

- inference FPS
- average latency
- p95/p99 latency where measured
- model execution time
- queue time
- resource usage

### Per model

- execution latency
- throughput

- memory footprint
- instance count

## Per device

- GPU utilization
- GPU memory
- DLA utilization where measurable
- decoder utilization where available
- thermal/health signals where available

These metrics are essential for optimizing 100+ camera deployments.

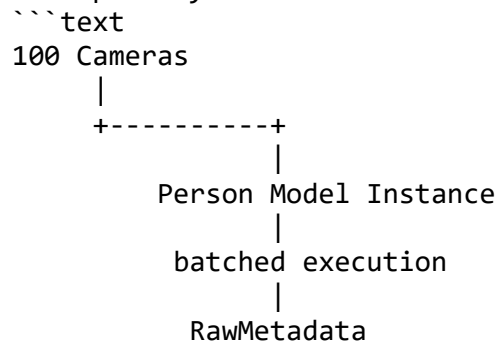
---

## 27. Model Sharing

Model sharing is a first-class optimization opportunity.

If multiple cameras use the same model and the runtime can safely batch/share execution, the platform should avoid one independent model instance per camera.

Conceptually:



---

However, sharing must not be forced when it causes unacceptable latency, state contamination, preprocessing incompatibility, or failure isolation problems.

---

## 28. Decode and Preprocessing Boundary

The Media Platform owns the video transport/streaming domain.

AI Runtime consumes the stream through the defined EncodedVideo contract and/or runtime integration path.

DeepStream should perform inference-oriented preprocessing where its supported components provide the required operation.

OpenCV/CV-CUDA are **not default preprocessing requirements**.

They may be used when a project/runtime requires custom image processing that is not appropriately handled by the standard DeepStream path.

Rule:

> Do not duplicate preprocessing outside DeepStream without a concrete technical reason.



---

## 29. GPU Optimization Principles

The runtime should continuously optimize for:

- GPU utilization
- latency
- throughput
- memory consumption
- decoder utilization
- accelerator utilization
- model sharing
- batching
- zero-copy paths
- frame skipping
- scheduling
- workload placement

Potential hardware acceleration may include, where supported by the deployment hardware:

- GPU
- DLA
- NVDEC
- VIC
- PVA
- other NVIDIA accelerators

Hardware-specific optimization must remain behind runtime abstractions so the platform remains portable across Jetson, RTX PC, and server deployments.

---

## 30. Multi-GPU / Multi-Node Direction

The platform should not assume one physical GPU.

The runtime architecture should allow a deployment planner to place applications/models onto available runtime resources.

Conceptually:

```text

Deployment

```
|
+--> GPU 0
|     +--> applications
|
+--> GPU 1
|     +--> applications
|
+--> DLA 0
|
+--> DLA 1
```

```

For multi-node systems, each node executes its assigned runtime workload while configuration, contracts, eventing, storage, and APIs remain platform-level concerns.

Exact distributed scheduling is intentionally not over-specified in v1.0.

---

## 31. Reusable Component Inventory

The AI Runtime Platform should ultimately contain reusable implementations for:

```
```text
ai\_runtime/
├── application\_registry/
├── application\_resolver/
├── model\_registry\_adapter/
├── model\_serving/
├── model\_loader/
├── model\_cache/
├── pipeline\_orchestrator/
├── topology\_planner/
├── execution\_engine/
├── scheduler/
├── resource\_manager/
├── batching/
├── gpu/
├── dla/
├── deepstream/
├── tensorrt/
├── metadata/
├── lifecycle/
├── health/
└── recovery/
```
```

The exact package structure may evolve. The capability boundaries are the reusable target.

---

## 32. Project Reuse Model

A new project must consume the AI Runtime rather than rebuild it.

Example:

```
```text
Existing CV Platform
|
+--> AI Runtime
|   +--> DeepStream
|   +--> Model Serving
|   +--> Scheduler
|   +--> Resource Manager
|   +--> Pipeline Orchestrator
|

```

```

      v
New Project
  |
  +--> configure cameras
  +--> select AI applications
  +--> select model versions
  +--> configure runtime policies
  +--> implement project-specific logic
...

```

Project-specific code must not duplicate the runtime's generic pipeline/model/scheduling infrastructure.

If a project requires a capability that is genuinely reusable, it should be promoted into AI Runtime rather than implemented as hidden project infrastructure.

33. Configuration Boundary

AI Runtime consumes configuration; it does not become the authoritative configuration database.

Configuration Platform owns:

- AI application definitions
- model configuration
- deployment configuration
- camera/application assignments
- runtime parameters

AI Runtime owns:

- resolved runtime state
- loaded models
- active pipelines
- runtime resource allocation
- execution state

This creates a clean distinction:

```
```text
```

```
Configuration Platform
```

```
"What should run?"
```

```
AI Runtime
```

```
"How is it executed?"
```

```
Runtime Health
```

```
"What is actually running?"
```

```
```
```

34. Security and Isolation

The runtime must treat model artifacts, configuration, and runtime commands as untrusted inputs until validated.

At minimum:

- validate model/runtime compatibility
- validate configuration before pipeline construction
- isolate application failures where practical
- avoid arbitrary shell execution from project configuration
- protect model and runtime credentials
- expose only intended runtime APIs

Security implementation details remain subject to the broader platform security architecture.

35. Architectural Laws for AI Runtime

Law 1

AI Runtime executes AI; it does not make business decisions.

Law 2

AI applications are the public abstraction; DeepStream pipeline topology is an implementation detail.

Law 3

Camera/application assignments are configuration, never hard-coded application logic.

Law 4

Models are versioned and executable through a reusable model-serving layer.

Law 5

Resource allocation is centralized; individual AI applications do not independently fight for GPU resources.

Law 6

RawMetadata is the primary outward AI contract.

Law 7

AI Runtime never directly sends alerts, notifications, GPIO commands, or other business actions.

Law 8

No project may duplicate generic AI Runtime infrastructure when the capability already exists in the platform.

Law 9

DeepStream-specific implementation must remain behind the runtime abstraction.

Law 10

Hardware-specific optimization must not destroy portability of the reusable runtime.

36. Non-Goals for v1.0

The following are deliberately not separate permanent platforms in v1.0:

- Model Manager Platform
- Scheduler Platform
- DeepStream Platform
- GPU Platform
- Preprocessing Platform

These are reusable components/capabilities inside AI Runtime or other established platforms.

They may become independent abstractions later only if real implementation pressure demonstrates that the existing boundary is insufficient.

37. Open Decisions Before Implementation Freeze

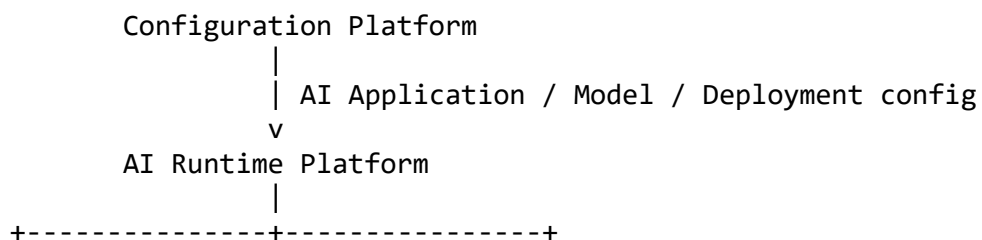
The following require engineering validation before being treated as immutable implementation rules:

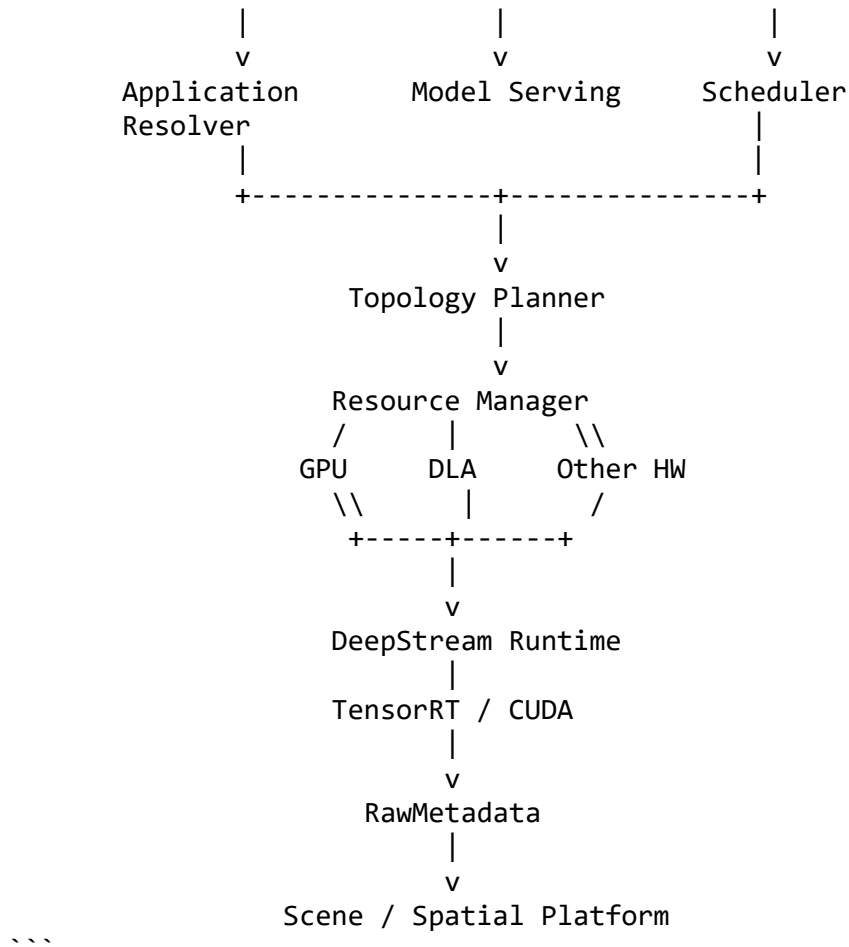
1. Exact DeepStream pipeline topology selection algorithm.
2. Conditions for shared model instances versus independent instances.
3. Cross-camera batching policy.
4. Scheduler algorithm and priority model.
5. GPU/DLA allocation algorithm.
6. Dynamic pipeline reconfiguration capabilities supported safely by the selected DeepStream version.
7. Exact model-serving mechanism and whether/when Triton or another serving layer is justified.
8. Exact multi-GPU and multi-node orchestration mechanism.
9. Exact metadata transport implementation and serialization format.
10. Hardware-specific use of VIC/PVA/NVDEC/OFA and how these capabilities are exposed to applications.

These are engineering decisions, not reasons to change the permanent 13-platform architecture.

38. Final Architecture

```text





The reusable objective is simple:

```
> **Build the AI execution machinery once. Future projects configure and
compose it; they do not rebuild it.**
```

---

## 39. Relationship to AIOS

AIOS is the development/orchestration layer. AI Runtime is the production CV execution subsystem.

AIOS may:

- discover this design
- select existing AI Runtime components
- generate project configuration
- invoke DeepStream-specific coding skills
- implement project-specific AI applications
- test and review runtime changes

AIOS must not silently redefine this architecture.

The AI Runtime Platform Design is authoritative for the internal AI Runtime architecture, while the global Architecture and Platform Contracts remain authoritative for cross-platform boundaries.